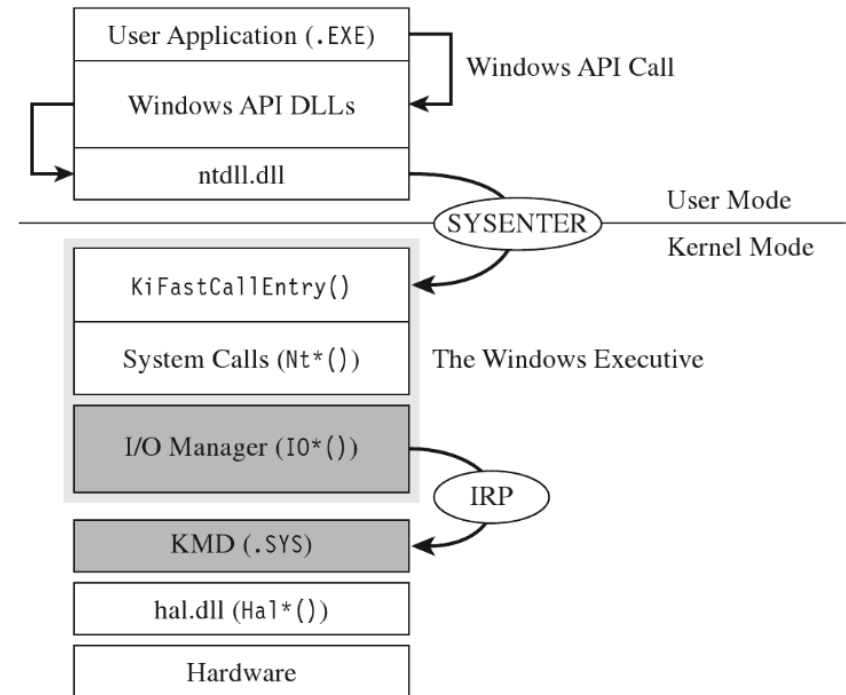
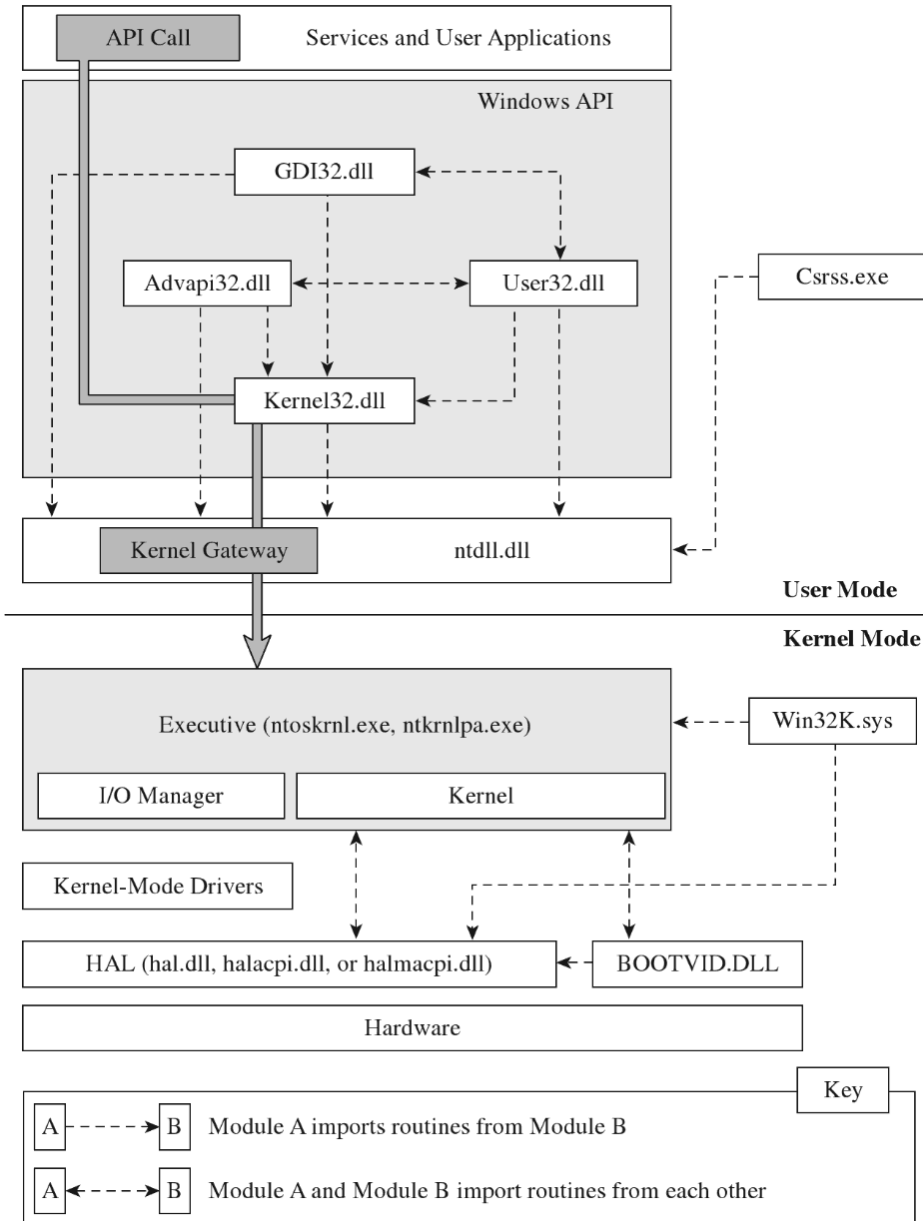


Sterowniki (sprzętowe ?)
w systemie Windows

Sterowniki systemu Windows KDM



Kroki do wykonania KDM

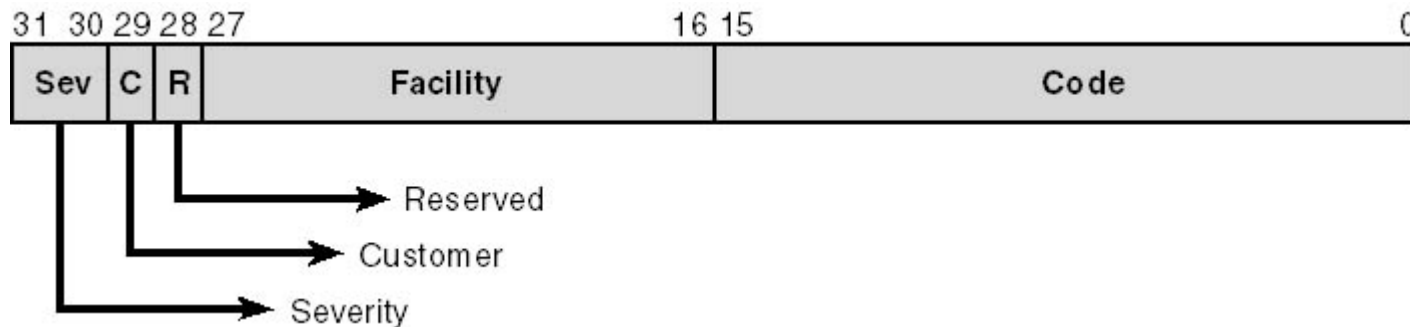
- Kod sterownika
- Kod aplikacji wywołującej funkcje sterownika
- Wymagany jest wspólny identyfikator, dzięki któremu aplikacja wyszuka sterownik (proces rejestracji obiektu sterownika w systemie)
- Sterownik trzeba zainstalować w systemie

Minimalny kod sterownika

```
#include "ntddk.h"
#include "dbgmsg.h"
VOID Unload (IN PDRIVER_OBJECT pDriverObject)
{
    DBG_TRACE("OnUnload", "Received signal to unload the driver");
    return;
}

NTSTATUS DriverEntry
(IN PDRIVER_OBJECT pDriverObject, IN PUNICODE_STRING regPath)
{
    DBG_TRACE("Driver Entry", "Driver has been loaded");
    (*pDriverObject).DriverUnload = Unload;
    return(STATUS_SUCCESS);
}
```

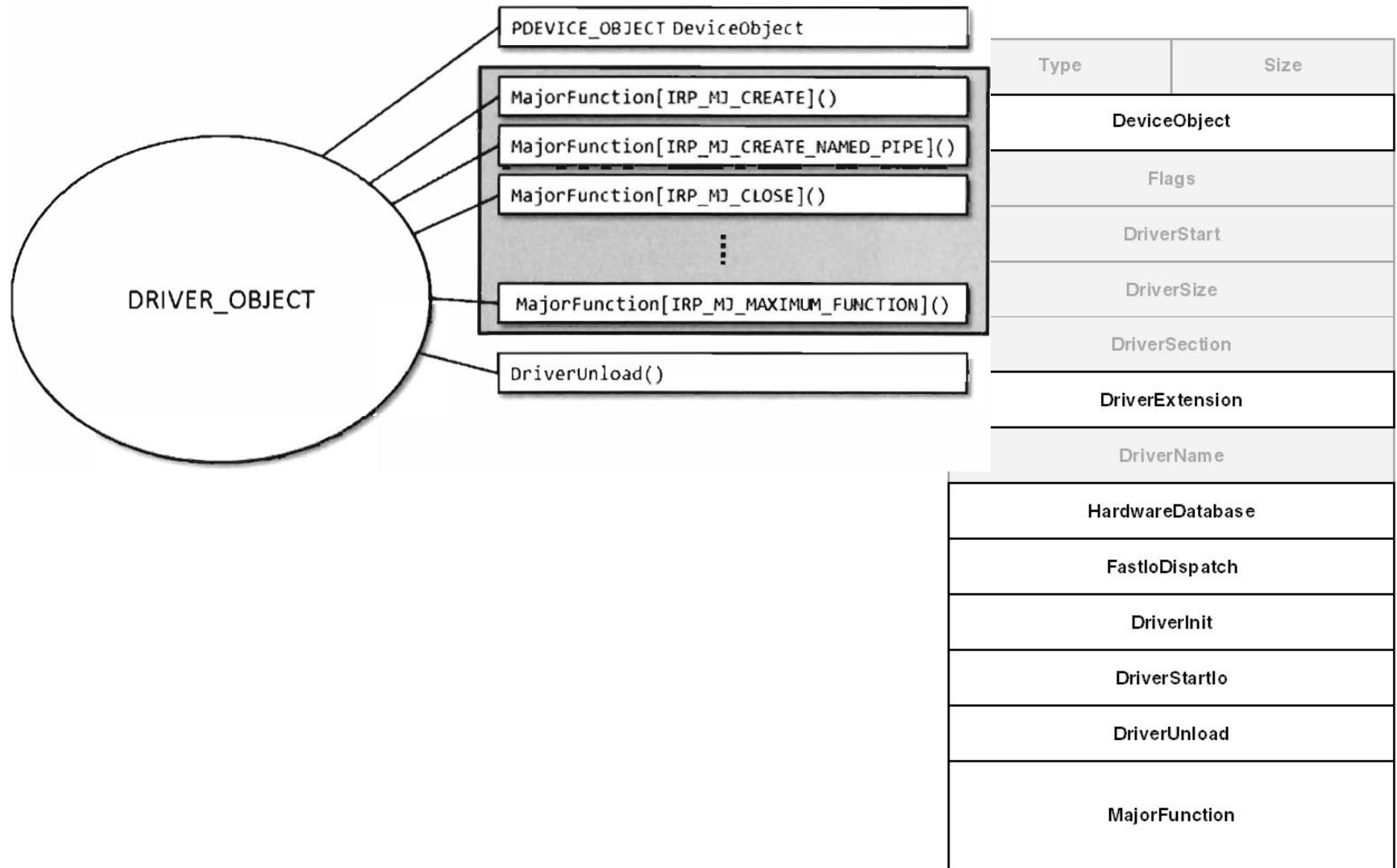
Kod powrotu



- **Sev** - kod powagi błędu
 - 00 - Success
 - 01 - Informational
 - 10 - Warning
 - 11 - Error
- **C** - jeśli wartość jest zdefiniowana przez użytkownika
- **Facility** - klasa błędu
- **Code** - Kod błędu

NT_SUCCESS(Status)
NT_INFORMATION(Status)
NT_WARNING(Status)
NT_ERROR(Status)

Struktura DRIVER_OBJECT



DeviceIoControl

```
(  
    hDeviceFile,  
    (DWORD)IOCTL_XXX_CMD,  
    (LPVOID)inBuffer,  
    nBufferSize,  
    (LPVOID)outBuffer,  
    nBufferSize,  
    &bytesRead,  
    NULL  
);
```

kernel32.dll

ntdll.dll

SYSENTER

User Mode

Kernel Mode

System Calls (Nt*())

I/O Manager (Io*())

IRP

```
inputBuffer      = (*pIRP).AssociatedIrp.SystemBuffer;  
outputBuffer     = (*pIRP).AssociatedIrp.SystemBuffer;  
irpStack         = IoGetCurrentIrpStackLocation(pIRP);  
inputBufferLength = (*irpStack).Parameters.DeviceIoControl.InputBufferLength;  
outputBufferLength = (*irpStack).Parameters.DeviceIoControl.OutputBufferLength;  
ioctrlcode       = (*irpStack).Parameters.DeviceIoControl.IoControlCode;
```

hal.dll (Hal*())

Hardware

Instalacja sterownika w systemie

- Wykorzystanie Service Control Manager'a
- Wykorzystanie wywołania `ZwSetSystemInformation()`
- Zapis do `\Device\PhysicalMemory`

SCM

```
@echo off
setlocal
set CREATE_OPTIONS= type= kernel start=
demand error= normal DisplayName=
driver3
sc.exe create srv3 binpath=
%windir%\System32\drivers\driver3.sys
%CREATE_OPTIONS%
sc.exe description driver3 "My OS Driver"
end local
```

ZwSetSystemInformation

- Wymagane jest pozyskanie adresu funkcji z ntdll.dll za pomocą `GetProcAddress`

```
ZwSetSystemInformation  
(LOAD_DRIVER_IMAGE_CODE,  
&DriverName,  
sizeof(DRIVER_NAME));
```

Ex - General executive routines
Exp - Executive private (not exported)
Cc - Cache Manager (Controller)
Mm - Memory Manager
Rtl - General runtime library
FsRtl - file system runtime library
Ob - object management
Io - I/O subsystem
Se - Security
Ps - Process structure
Po - Power management
Wmi - Windows Management Instrumentation
Zw - File and registry access
Ke - General Kernel
Ki - Kernel internal (not available outside of kernel)
Hal - hardware abstraction layer
READ_xxx, WRITE_xxx - I/O port and register access (HAL)

Nazwy obiektów

- Użytkownik widzi tylko obiektu w katalogach `\BaseNamedObjects` i `\??`
 - Obiekty rezydujące w `\BaseNamedObjects` są globalne i tworzone przez aplikacje użytkownika
 - KDM umieszcza łącza symboliczne w `\??`
- Katalog wszystkich obiektów
 - Struktura zarządzana przez Object Manager'a.
 - Tworzenie obiektów w katalogu z poziomu kernela za pomocą *`ZwCreateDirectoryObject`*
- Łącza symboliczne
 - `\??\A:` `\Device\Floppy0`
 - `\??\COM1` `\Device\Serial0`
 - Dostęp z poziomu użytkownika *`QueryDosDevice`*